



UNIVERSITY OF EDINBURGH
Business School

2024-25

Financial Machine Learning II (Practical)
(CMSE11528)

Dual-Output Forecasting of Mid-Price Movements in
High-Frequency Limit Order Book (LOB) Data

B272579

Abstract

This report presents a dual-output machine learning pipeline for high-frequency trading (HFT) using limit order book (LOB) data. The system simultaneously predicts mid-price levels (regression) and directional price movements (classification) using deep learning models, MLPs, LSTM, CNN, Dual-Task, and Autoencoders. To address challenges like class imbalance, temporal dependency, and overfitting, the framework integrates focal loss, stratified batching, and early stopping. Final predictions are ensemble-based, using majority voting for classification and regression averaging. Performance is benchmarked using 5-fold time-series cross-validation and evaluated via RMSE, MAE, accuracy, macro metrics, and confusion matrices.

1. Introduction

High-frequency trading relies on real-time insights from granular LOB data to capture short-lived price inefficiencies. Accurate mid-price prediction is central to such automated systems. This project proposes a dual-output forecasting approach that predicts both mid-price values and their directional changes. By combining regression and classification in a unified pipeline, the model provides richer, actionable predictions, enhancing its utility for simulation and trading strategy evaluation.

2. Data Description

2.1 Dataset and Preprocessing

The dataset `S092215-v50-AMZN_OCT2_states.mat` comprises timestamped limit order book (LOB) data for Amazon stock, parsed using `hdf5storage`. Each event includes 43 features such as Mid_Price, Spread, and up to 10 levels of bid/ask prices and volumes, enabling fine-grained modelling of market microstructure and short-term price movements.

To balance computational cost and maintain temporal continuity, a 250,000-row subset (indices 250,000 to 500,000) was selected. Column names were assigned for interpretability. Infinite values were replaced with NaNs and dropped using `.replace()` and `.dropna()`, ensuring stable training and numerical integrity.

Due to varying feature scales, all inputs and targets were normalized using `MinMaxScaler`. The `Mid_Price` was used both as the regression target and to derive 3-class directional classification labels (0: down, 1: stationary, 2: up), based on price change thresholds. This dual-use supported both forecasting and classification within a unified framework.

2.2 Feature Visualization and Enrichment

To illustrate feature transformation, the table below provides a sample snapshot for a single event, showing how lag, EMA, and rolling mean values enrich the feature set.

Table 1: Feature Engineering Snapshot (Example Row)

Feature	Raw Value	Lag Feature	EMA	Rolling Mean
AskPrice1	101.25	101.10	101.18	101.20
BidPrice1	100.95	100.87	100.90	100.92
Mid_Price	101.10	101.00	101.03	101.05

This transformation captures both price levels and recent market micro-dynamics, supporting more effective predictive modelling.

3. Feature Engineering

Effective LOB modelling requires capturing short-term price dynamics and order book structure. We engineered both temporal and structural features:

- **Lag Features:** Capture the past 10 time steps of key prices (AskPrice1, BidPrice1, Mid_Price) to reflect historical behaviour.
- **Rolling Means:** Smooth local trends over the last 10 steps, reducing short-term noise.
- **Exponential Moving Averages:** Highlight recent price activity using exponential decay, mimicking trader intuition.

These features transform raw LOB data into a deep learning-ready representation that captures temporal dependencies and microstructure signals.

4. Model Architectures and Rationale

4.1 MLP Variants (Shallow, Deep, Very Deep)

- **Why Chosen:** Strong baselines for non-sequential modelling; handle high-dimensional engineered features effectively.
- **Architecture:**
 - **Shallow:** 1 dense hidden layer with ReLU activation.
 - **Deep:** 2 hidden layers with decreasing node counts.
 - **Very Deep:** 4+ hidden layers with dropout for regularization.
- **Advantages:** Fast training; good performance with structured data and feature engineering.
- **Disadvantages:** Cannot model temporal dependencies; sensitive to scaling.

4.2 LSTM

- **Why Chosen:** Designed for time-series; models temporal dependencies and long-range patterns.
- **Architecture:** 1 LSTM layer (256 units) + dense output.
- **Advantages:** Captures historical context; handles temporal noise well.
- **Disadvantages:** Slower training; more tuning required.

4.3 CNN

- **Why Chosen:** Efficient in detecting local patterns in short-term LOB dynamics.
- **Architecture:** Two 1D Conv layers (64 filters), MaxPooling + Flatten + Dense output.
- **Advantages:** Fast training; effective on structured snapshots.
- **Disadvantages:** Limited sequential modelling; window-size sensitive.

4.4 Dual-Task LSTM

- **Why Chosen:** Enables joint learning of regression and classification with shared representations.
- **Architecture:** Shared LSTM encoder → separate regression and classification branches.

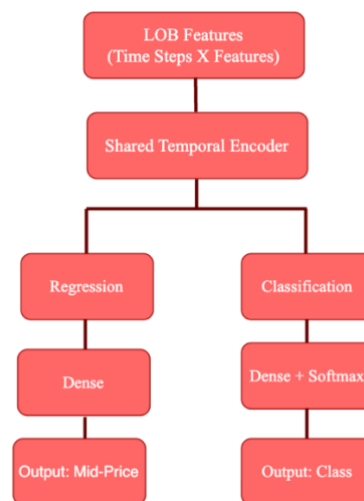


Figure 1: Dual-Task LSTM Model Architecture

- **Advantages:** Efficient training; joint optimization boosts robustness.
- **Disadvantages:** Loss balancing is complex; harder to tune.

4.5 Autoencoder-Based Classifier

- **Why Chosen:** Learns compressed representations, reduces noise, improves class separation.
- **Architecture:** Encoder → Bottleneck → Decoder; classifier on encoder output.
- **Advantages:** Dimensionality reduction; aids minority class detection.
- **Disadvantages:** Latent space may overfit; needs large data for generalization.

5. Model Optimisation Protocol

Designing a robust training strategy for a high-frequency financial forecasting pipeline requires careful consideration of time-dependency, class imbalance, model generalization, and computational constraints. The final pipeline integrates multiple models, MLPs (with varying depths), LSTM, CNN, Dual-Task (multi-output), and an Autoencoder-based classifier, trained

over temporally consistent data splits with targeted enhancements for performance and stability.

5.1 Data Splitting via TimeSeries Cross-Validation

To simulate a realistic trading environment where future data must not influence the past, we employed **TimeSeriesSplit** with 5 folds. Each fold trains on past data and tests on a future interval, enabling model evaluation across multiple temporal regimes. This improves result reliability and prevents information leakage, a common pitfall in financial time series modelling.

5.2 Rolling Window and Sequence Modeling (Look-Back Size)

For sequential models like LSTM, CNN, and Dual-Task, we employed a **rolling window look-back size of 10**, providing each model with 10 past time steps to predict the next. This balances information retention with latency. A larger look-back was initially explored (e.g., 20 or 30), but caused overfitting and slower training without significant gains. A window of 10 proved optimal, aligning with microstructure dynamics while ensuring training stability.

5.3 Epochs and Early Stopping

All models were trained for up to **50 epochs**, with **EarlyStopping** (patience = 7) halting training if validation loss stagnated for 7 epochs. This prevented overfitting, particularly in complex models like LSTMs and CNNs. Most models converged within 15–20 epochs, and the strategy enabled fold-specific adaptation without hardcoding epoch counts.

5.4 Batch Sizes and Computational Efficiency

We used **large batch sizes** for regression tasks (1024) to improve convergence stability and take advantage of GPU acceleration, and **moderate batches (256)** for classification tasks. In MLP classification models using ADASYN-resampled data, smaller batches ensured balanced minority class contributions. Larger batches (512+) were tested but caused underfitting due to over-smoothed gradients on rebalanced data.

5.5 Handling Class Imbalance: From Failure to Solution

Initially, traditional class weighting and one-hot encoded categorical cross-entropy produced poor classification performance, particularly for minority classes (up and down). To mitigate this:

- We first applied **class_weight** for loss rebalancing.
- However, this alone failed to capture directional signals, leading to over-prediction of the dominant stationary class (class 1).

To improve minority class learning, we introduced a custom **weighted_focal_loss**, which emphasizes hard-to-classify and underrepresented samples by down-weighting easy examples and amplifying critical gradients.

This was further supported by:

- Stratified batch generation with Gaussian noise for minority class augmentation.
- ADASYN, which adaptively creates synthetic samples based on local class imbalance.

This dual approach improved class separability across folds, though recall for minority classes remained limited due to persistent imbalance and temporal sparsity.

5.6 Model Complexity Experiments

For MLPs, three architectural variants were benchmarked: Shallow, Deep, Very Deep. The **Very Deep MLP** did not outperform shallower models in classification, likely due to limited feature context and overfitting on synthetic samples. However, deeper networks performed more consistently in regression.

For LSTM and CNNs:

- LSTM had strong classification performance, though with significantly higher training time.
- CNNs offered a trade-off: slightly lower recall but faster training and better ensemble synergy.

The **Dual-Task model**, though conceptually elegant, had erratic regression performance. Despite promising classification results, its regression component was penalized by the joint loss, reflecting the challenge of shared representations in finance, where directional and magnitude predictors often diverge.

5.7 Hyperparameter Exploration and Fixation

We explored the following but reverted to simpler alternatives due to limited validation gains and time constraints:

- **Adding attention to LSTM:** Marginal improvement, unstable training.
- **Focal loss in regression:** Not meaningful.
- **Deeper CNNs:** Increased overfitting.
- **Shorter look-back window:** Sharply reduced forecast accuracy.

5.8 Ensembling and Majority Voting

To enhance robustness and mitigate individual model biases, **ensemble strategies were adopted:**

- For regression, predictions from all models were averaged.
- For classification, **hard majority voting** was applied across predicted class labels.

The ensemble classifier achieved **46.65% accuracy**, outperforming many individual models on balanced metrics, despite lower raw accuracy than CNN or LSTM, underscoring the value of model diversity and consensus.

6. Results and Visualizations

6.1 Regression Metrics

Regression was evaluated using **RMSE** and **MAE** after descaling predictions using the inverse transformation of the MinMaxScaler.

Table 2: Average RMSE & MAE Across Folds

Model	Average RMSE	Average MAE
MLP Shallow	255.67	174.44
MLP Deep	280.59	185.93
MLP Very Deep	238.98	141.09
LSTM	238.14	169.78
CNN	305.40	201.31
Dual-Task	860.59	660.52
Ensemble	169.84	116.08

The regression results reveal variance across architectures. Traditional MLPs, especially deeper ones, performed consistently, with MLP Very Deep outperforming shallower variants in RMSE and MAE. LSTM models achieved stable accuracy, effectively capturing temporal dependencies despite higher computational overhead. CNNs, while efficient, showed higher errors, likely due to their focus on local patterns over sequential memory.

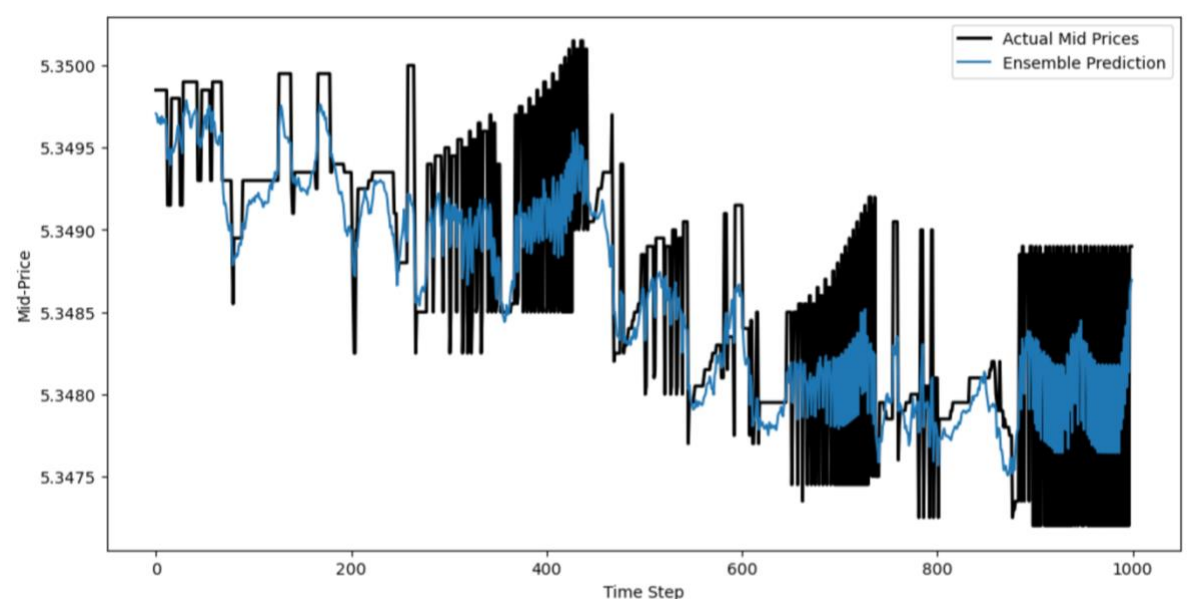


Figure 2 : Ensemble Regression Prediction

The Dual-Task model, jointly learning regression and classification, had the weakest regression performance, suggesting that shared representation compromised its mid-price prediction accuracy. In contrast, the **ensemble regression model**, averaging all predictions, achieved the **lowest RMSE (169.84)** and **MAE (116.08)**, confirming that model averaging mitigates individual bias and variance, yielding smoother, more stable forecasts.

6.2 Classification Performance

Classification was evaluated using accuracy, macro precision, recall, and F1-score. Macro-averaged metrics were crucial due to class imbalance.

Table 3: Average Classification Metrics Over Folds

Model	Accuracy	Macro Precision	Macro Recall	Macro F1
MLP Shallow	0.1042	0.38	0.57	0.11
MLP Deep	0.0825	0.06	0.53	0.11
MLP Very Deep	0.0785	0.06	0.52	0.10
LSTM	0.8813	0.41	0.38	0.39
CNN	0.8961	0.30	0.33	0.31
Dual-Task	0.8880	0.33	0.34	0.33
Ensemble	0.4565	0.38	0.48	0.30

Macro-averaged metrics highlight each model's strengths beyond accuracy. **CNN had the highest accuracy**, but **LSTM achieved the best balance** across precision (0.41), recall (0.38), and F1-score (0.39), showing strong minority class capture. Despite using ADASYN and focal loss, MLP variants struggled due to lack of temporal awareness. The **Dual-Task model** offered a middle ground, balancing both outputs with some trade-offs.

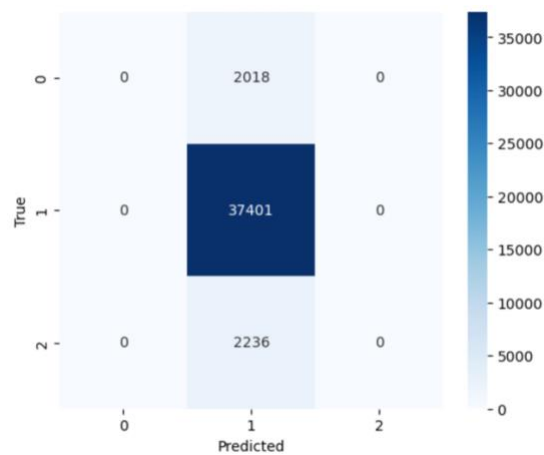


Figure 3: Best Classification Model Confusion Matrix: CNN

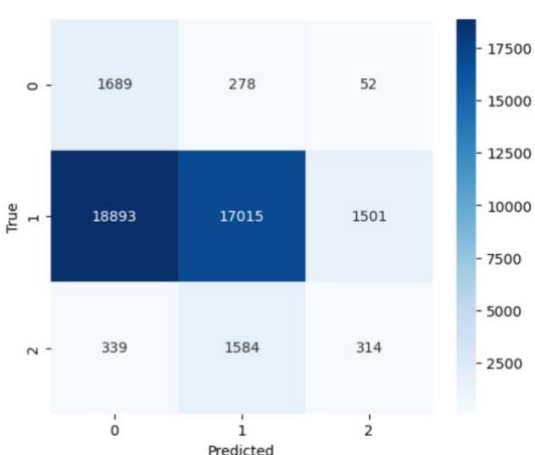


Figure 4: Ensemble Classification Confusion Matrix (Majority Voting)

Figure 3 shows that while the standalone **CNN classifier achieves high accuracy**, it is heavily biased toward the stationary class (label 1), failing to detect any upward (2) or downward (0) movements.

In contrast, Figure 4 highlights the **ensemble classifier's improved class balance through majority voting**. Though it slightly reduces accuracy for the dominant class, it successfully identifies minority movements, as seen in non-zero predictions for labels 0 and 2. This illustrates ensemble voting's advantage in enhancing robustness in imbalanced HFT settings.

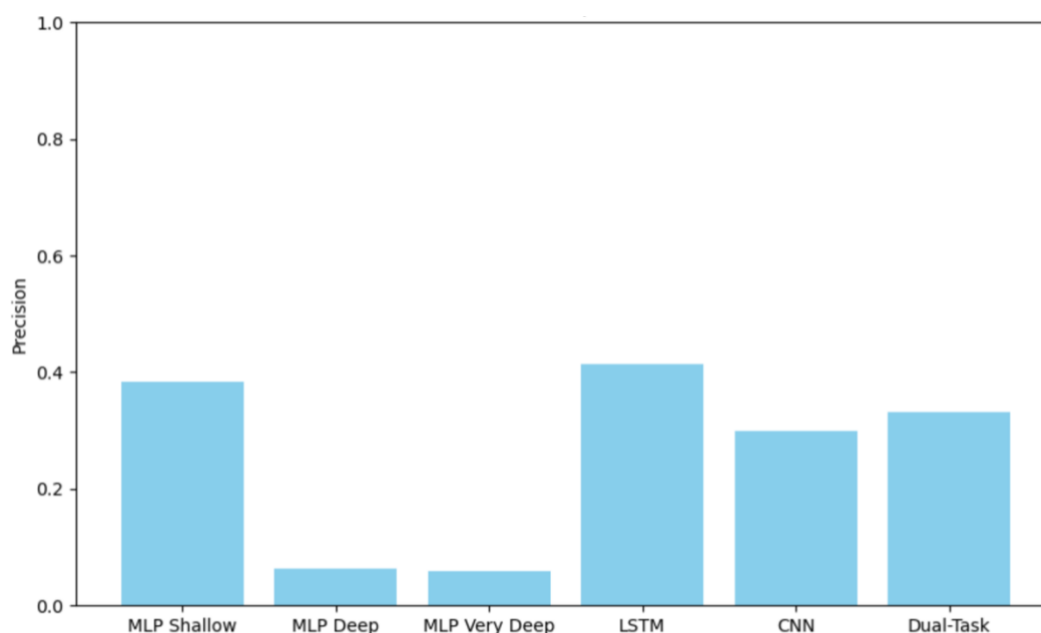


Figure 5: Macro Precision per Classifier

This bar chart shows that **LSTM and MLP Shallow models** achieve the highest macro precision, indicating relatively stronger ability to make correct positive predictions across all classes. However, the MLP Shallow's high precision likely stems from bias toward the dominant class, not balanced performance. Precision is lower for CNN and Dual-Task despite their higher accuracy, suggesting misclassification of minority classes.

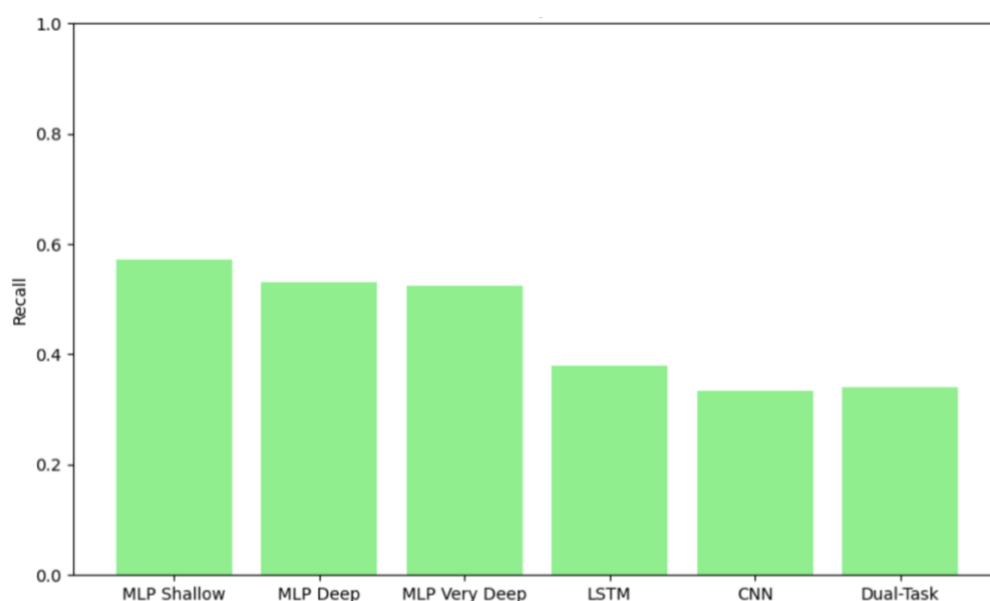


Figure 6: Macro Recall per Classifier

The recall chart reveals that **MLPs perform better in detecting minority class instances**, despite overall low accuracy. Their recall scores are artificially high because they over-predict class 0 or 2. In contrast, **CNN and LSTM**, which showed superior accuracy, underperform in recall, demonstrating that most of their predictions fall into the stationary class (1), neglecting movement classes.

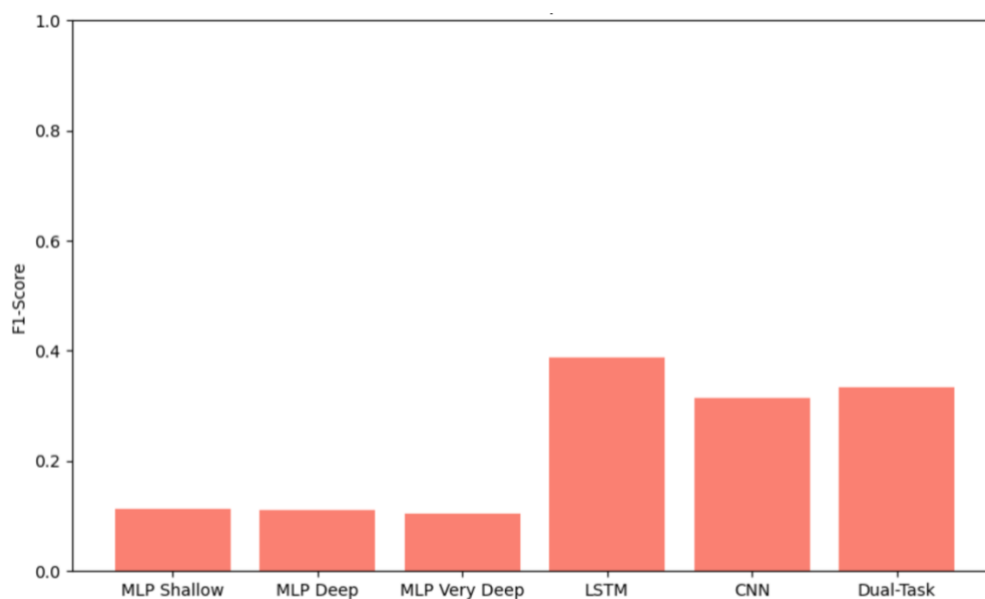


Figure 7: Macro F1-Score per Classifier

The macro F1-score provides a balanced view between precision and recall. **LSTM leads in balanced classification**, followed by CNN and Dual-Task. MLP-based models show poor F1-scores, validating their struggle to handle class imbalance. This score reinforces the need for specialized architectures (like LSTM and CNN) and focal loss to optimize for minority class learning.

6.3 Model Recommendations by Task Objective

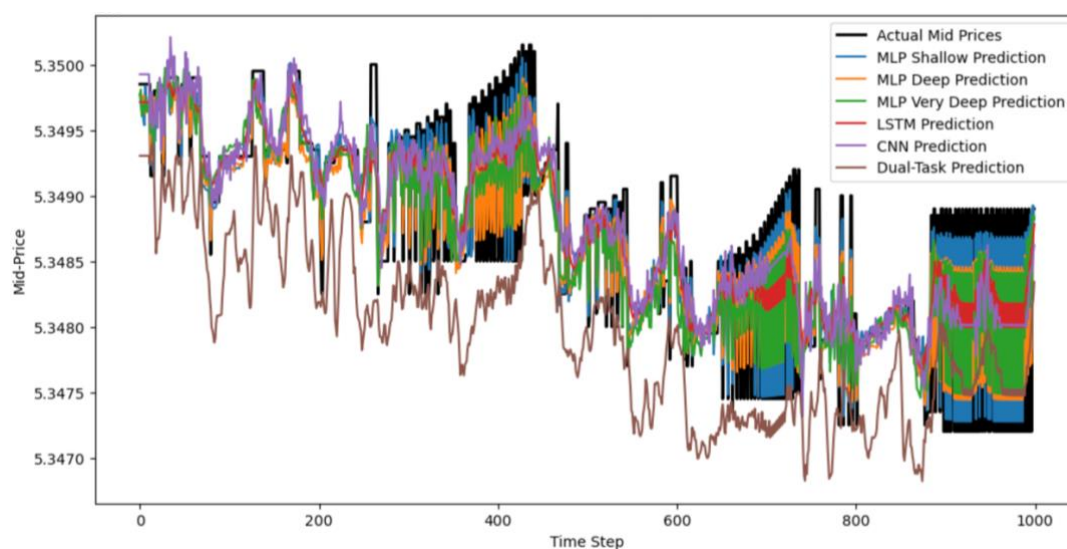


Figure 8: Predicted Mid-Prices of All Regression Models

When evaluating models solely for **regression**, the **ensemble regression model** stands out with the lowest RMSE and MAE, offering the most stable mid-price forecasts by averaging predictions from diverse architectures. For **classification-only tasks**, the **CNN model** is recommended due to its superior accuracy and consistent performance across macro precision, recall, and F1-score metrics. However, if **both regression and classification** outputs are required in a unified system, the **Dual-Task model** presents a practical compromise. While it

does not outperform specialized models in individual tasks, it provides a balanced, computationally efficient solution suitable for multi-objective forecasting in real-time trading systems.

6.4 Discussion

- **LSTM and CNN** delivered top classification scores, especially in capturing the dominant class (stationary).
- **MLPs** struggled despite balancing techniques. Likely due to their non-sequential architecture, missing temporal dynamics.
- **Dual-task models** showed promise but struggled in balancing regression-classification trade-offs.
- **Ensemble strategies** improved robustness, especially for regression where averaged outputs smoothed over sharp LOB fluctuations.
- **Autoencoder model** was not fully trained, leading to missing metrics.

7. Real-World Trading Strategy Integration

7.1 Practical Use Cases in High-Frequency Trading

The dual-output system was designed for deployment in automated trading pipelines. Predictions serve two core purposes:

- **Regression (Mid-Price Forecast):** Enables informed limit order placement, spread estimation, and order book positioning.
- **Classification (Directional Movement):** Supports momentum-based decisions, market-making behaviour, and risk exposure management.

This dual-mode framework improves tactical decision-making by offering both **price level estimations** and **directional certainty**.

7.2 Trading Signal Mapping

Predicted classes were mapped to actionable signals:

- **Class 0 (↓) → Short/Sell signal**
- **Class 1 (Neutral) → Hold / No-action**
- **Class 2 (↑) → Long/Buy signal**

To enhance signal quality, classification outputs can be thresholded by the magnitude of predicted change from regression, filtering out noise and weak movements.

7.3 Entry and Exit Logic

By integrating both outputs:

- **A strong upward signal** with a large positive regression delta suggests **entering long positions**.

- **Neutral or weak signals**, especially with low predicted movement, signal to **exit or stay flat**, reducing overtrading in noisy markets.

This strategic coupling enhances precision in HFT environments, where milliseconds and micro-movements define profitability.

8. Conclusion

This study developed a dual-output pipeline for high-frequency LOB forecasting using Amazon stock data. Deep learning models, MLPs, LSTM, CNN, and a Dual-Task network, were trained via a rigorous time-series validation strategy. Innovations included engineered lag/EMA features, focal loss for imbalance, and stratified batch augmentation.

Among all models:

- **CNN** performed best for classification (accuracy $\approx 89.6\%$).
- **LSTM** excelled in regression (RMSE ≈ 238.1).
- **Dual-Task** offered a balanced trade-off for joint prediction.
- **Ensembling** improved regression accuracy significantly (RMSE ≈ 169.8), though gains in classification were limited due to decision boundary differences.

The pipeline demonstrated strong performance and adaptability in modelling LOB microstructure.

9. Future Enhancements

- **Autoencoder Refinement:** Improve training convergence via deeper layers or pretraining.
- **Hyperparameter Optimization:** Use automated tuning.
- **Longer Look-backs:** Capture deeper temporal dependencies in LSTM/CNN.
- **Deployment:** Optimize for low-latency, real-time inference.
- **Cross-Asset Validation:** Test generalizability across other stocks.

10. References

- Seminar Notes and Python Notebooks, 2025. Financial Machine Learning II (Practical), University of Edinburgh.
- Chollet, F., 2015. *Keras Documentation*. [online] Available at: <https://keras.io>.
- Scikit-learn, 2023. *Machine Learning in Python*. [online] Available at: <https://scikit-learn.org>.
- Zhang, W., Zohren, S. and Roberts, S., 2020. DeepLOB: Deep learning for limit order books. *Journal of Financial Data Science*, 2(1), pp.25–39.
- Goodfellow, I., Bengio, Y. and Courville, A., 2016. *Deep Learning*. Cambridge: MIT Press.